

如何使用 Cloud Run 工作微調 LLM

來源：<https://codelabs.developers.google.com/codelabs/cloud-run/how-to-fine-tune-model-cloud-run-jobs?hl=zh-tw>

步驟數：9

瞭解如何使用 Cloud Run 工作微調模型

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 設定和需求](#)
4. [4. 建立 Cloud Run 工作映像檔](#)
5. [5. 部署及執行工作](#)
6. [6. 使用 Cloud Run 服務，透過 vLLM 提供微調模型](#)
7. [7. 測試微調模型](#)
8. [8. 恭喜！](#)
9. [9. 清除所用資源](#)

步驟 1 · 約 0 分鐘

1. 簡介

總覽

在本程式碼實驗室中，您將使用 Cloud Run 作業微調 [Gemma 3](#) 模型，然後使用 [vLLM](#) 在 Cloud Run 上提供結果。

學習內容

使用 [KomeijiForce/Text2Emoji](#) 資料集訓練模型，針對特定片語產生特定結果。這個資料集是 [EmojiLM：為新表情符號語言建立模型](#)的一部分。

訓練完成後，模型會回應以「Translate to emoji:」為前置字元的句子，並提供與該句子相應的一連串表情符號。

課程內容

- 如何使用 Cloud Run Jobs GPU 進行微調
 - 如何使用 Cloud Run 和 vLLM 提供模型服務
 - 如何為 GPU 工作使用直接虛擬私有雲設定，加快模型上傳和服務速度
-

步驟 2 · 約 0 分鐘

2. 事前準備

啟用 API

開始使用本程式碼研究室前，請先執行下列指令，啟用下列 API：

```
gcloud services enable run.googleapis.com \  
compute.googleapis.com \  
run.googleapis.com \  
cloudbuild.googleapis.com \  
secretmanager.googleapis.com \  
artifactregistry.googleapis.com
```

GPU 配額

請參閱 [GPU 配額](#) 說明文件，確認如何要求配額。

如果遇到「您沒有使用 GPU 的配額」錯誤，請前往 g.co/cloudrun/gpu-quota 確認配額。

注意：如果您使用新專案，啟用 API 後，配額可能需要幾分鐘才會顯示在配額頁面。

Hugging Face

本程式碼研究室使用 [Hugging Face](#) 上代管的模型。如要取得這個模型，請申請具備「讀取」權限的 [Hugging Face 使用者存取權杖](#)。您稍後會以 `YOUR_HF_TOKEN` 參照這個位址。

如要使用 [gemma-3-1b-it 模型](#)，請務必同意使用條款。

3. 設定和需求

設定下列資源：

- IAM 服務帳戶和相關聯的 IAM 權限，
 - [Secret Manager 密鑰](#)，用於儲存 Hugging Face 權杖
 - 儲存微調模型的 Cloud Storage 值區，以及
 - Artifact Registry 存放區，用於儲存您將建構的映像檔，以微調模型。
1. 為本程式碼研究室設定環境變數。我們已為您預先填入多個變數。指定專案 ID、地區和 Hugging Face 權杖。

```
export PROJECT_ID=<YOUR_PROJECT_ID>
export REGION=<YOUR_REGION>
export HF_TOKEN=<YOUR_HF_TOKEN>

export NEW_MODEL=gemma-emoji
export AR_REPO=codelab-finetuning-jobs
export IMAGE_NAME=finetune-to-gcs
export JOB_NAME=finetuning-to-gcs-job
export BUCKET_NAME=$PROJECT_ID-codelab-finetuning-jobs
export SECRET_ID=HF_TOKEN
export SERVICE_ACCOUNT="finetune-job-sa"
export SERVICE_ACCOUNT_ADDRESS=$SERVICE_ACCOUNT@$PROJECT_ID.iam.gserviceaccount.com
```

2. 執行下列指令來建立服務帳戶：

```
gcloud iam service-accounts create $SERVICE_ACCOUNT \
  --display-name="Service account for fine-tuning codelab"
```

3. 使用 Secret Manager 儲存 Hugging Face 存取權杖：

```
gcloud secrets create $SECRET_ID \
  --replication-policy="automatic"

printf $HF_TOKEN | gcloud secrets versions add $SECRET_ID --data-file=-
```

4. 將 Secret Manager 密鑰存取者角色授予服務帳戶：

```
gcloud secrets add-iam-policy-binding $SECRET_ID \
  --member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \
  --role='roles/secretmanager.secretAccessor'
```

5. 建立值區，用於代管微調模型：

```
gcloud storage buckets create -l $REGION gs://$BUCKET_NAME
```

6. 授予服務帳戶 bucket 存取權：

```
gcloud storage buckets add-iam-policy-binding gs://$BUCKET_NAME \  
  --member=serviceAccount:$SERVICE_ACCOUNT_ADDRESS \  
  --role=roles/storage.objectAdmin
```

7. 建立 Artifact Registry 存放區來儲存容器映像檔：

```
gcloud artifacts repositories create $AR_REPO \  
  --repository-format=docker \  
  --location=$REGION \  
  --description="codelab for finetuning using CR jobs" \  
  --project=$PROJECT_ID
```

步驟 4 · 約 0 分鐘

4. 建立 Cloud Run 工作映像檔

在下一個步驟中，您將建立執行下列動作的程式碼：

- 從 Hugging Face 匯入 Gemma 模型
- 使用 Hugging Face 的資料集微調模型。這項工作使用單一 L4 GPU 進行微調。
- 將名為 `new_model` 的微調模型上傳至 Cloud Storage bucket

1. 為微調工作程式碼建立目錄。

```
mkdir codelab-finetuning-job  
cd codelab-finetuning-job
```

2. 建立名為 `finetune.py` 的檔案

```

# Copyright 2025 Google LLC
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#     http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.

import os

import torch
from datasets import load_dataset
from peft import LoraConfig, PeftModel
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    BitsAndBytesConfig,
    TrainingArguments,
)
from trl import SFTTrainer

# Cloud Storage bucket to upload the model
bucket_name = os.getenv("BUCKET_NAME", "YOUR_BUCKET_NAME")

# The model that you want to train from the Hugging Face hub
model_name = os.getenv("MODEL_NAME", "google/gemma-3-1b-it")

# The instruction dataset to use
dataset_name = "KomeijiForce/Text2Emoji"

# Fine-tuned model name
new_model = os.getenv("NEW_MODEL", "gemma-emoji")

##### Setup #####

# Load the entire model on the GPU 0
device_map = {"": torch.cuda.current_device()}

# Limit dataset to a random selection
dataset = load_dataset(dataset_name, split="train").shuffle(seed=42).select(range(1000))

# Setup input formats: trains the model to respond to "Translate to emoji:" with emoji
output.
tokenizer = AutoTokenizer.from_pretrained(model_name)

def format_to_chat(example):

```



```

    return {
        "conversations": [
            {"role": "user", "content": f"Translate to emoji: {example['text']}"},
            {"role": "assistant", "content": example["emoji"]},
        ]
    }

formatted_dataset = dataset.map(
    format_to_chat,
    batched=False,
    remove_columns=dataset.column_names, # Optional: Keep only the new column
)

def apply_chat_template(examples):
    texts = tokenizer.apply_chat_template(examples["conversations"], tokenize=False)
    return {"text": texts}

final_dataset = formatted_dataset.map(apply_chat_template, batched=True)

##### Config #####

# Load tokenizer and model with QLoRA configuration
bnb_4bit_compute_dtype = "float16" # Compute dtype for 4-bit base models
compute_dtype = getattr(torch, bnb_4bit_compute_dtype)

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True, # Activate 4-bit precision base model loading
    bnb_4bit_quant_type="nf4", # Quantization type (fp4 or nf4)
    bnb_4bit_compute_dtype=compute_dtype,
    bnb_4bit_use_double_quant=False, # Activate nested quantization for 4-bit base
models (double quantization)
)

# Load base model
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map=device_map,
    torch_dtype=torch.float16,
)
model.config.use_cache = False
model.config.pretraining_tp = 1

##### Train #####

# Load LoRA configuration
peft_config = LoraConfig(
    lora_alpha=16, # Alpha parameter for LoRA scaling
    lora_dropout=0.1, # Dropout probability for LoRA layers,
    r=8, # LoRA attention dimension
    bias="none",
    task_type="CAUSAL_LM",

```

```

        target_modules=["q_proj", "v_proj"],
    )

# Set training parameters
training_arguments = TrainingArguments(
    output_dir="./results",
    num_train_epochs=1,
    per_device_train_batch_size=1, # Batch size per GPU for training
    gradient_accumulation_steps=2, # Number of update steps to accumulate the gradients
    for
        optim="paged_adamw_32bit",
        save_steps=0,
        logging_steps=5,
        learning_rate=2e-4, # Initial learning rate (AdamW optimizer)
        weight_decay=0.001, # Weight decay to apply to all layers except bias/LayerNorm
    weights
        fp16=True, bf16=False, # Enable fp16/bf16 training
        max_grad_norm=0.3, # Maximum gradient normal (gradient clipping)
        warmup_ratio=0.03, # Ratio of steps for a linear warmup (from 0 to learning rate)
        group_by_length=True, # Group sequences into batches with same length # Saves memory
    and speeds up training considerably
        lr_scheduler_type="cosine",
    )

trainer = SFTTrainer(
    model=model,
    train_dataset=final_dataset,
    peft_config=peft_config,
    dataset_text_field="text",
    max_seq_length=512, # Maximum sequence length to use
    tokenizer=tokenizer,
    args=training_arguments,
    packing=False, # Pack multiple short examples in the same input sequence to
    increase efficiency
)

trainer.train()
trainer.model.save_pretrained(new_model)

##### Save #####

# Reload model in FP16 and merge it with LoRA weights
base_model = AutoModelForCausalLM.from_pretrained(
    model_name,
    low_cpu_mem_usage=True,
    return_dict=True,
    torch_dtype=torch.float16,
    device_map=device_map,
)

model = PeftModel.from_pretrained(base_model, new_model)
model = model.merge_and_unload()

```

```
# push results to Cloud Storage
file_path_to_save_the_model = "/finetune/new_model"
model.save_pretrained(file_path_to_save_the_model)
tokenizer.save_pretrained(file_path_to_save_the_model)
```

3. 建立 requirements.txt 檔案：

```
accelerate==0.34.2
bitsandbytes==0.45.5
datasets==2.19.1
transformers==4.51.3
peft==0.11.1
trl==0.8.6
torch==2.3.0
```

4. 建立 Dockerfile：

```
FROM nvidia/cuda:12.6.2-runtime-ubuntu22.04

RUN apt-get update && \
    apt-get -y --no-install-recommends install python3-dev gcc python3-pip git && \
    rm -rf /var/lib/apt/lists/*

COPY requirements.txt /requirements.txt

RUN pip3 install -r requirements.txt --no-cache-dir

COPY finetune.py /finetune.py

ENV PYTHONUNBUFFERED 1

CMD python3 /finetune.py --device cuda
```

5. 在 Artifact Registry 存放區中建構容器：

```
gcloud builds submit \
  --tag $REGION-docker.pkg.dev/$PROJECT_ID/$AR_REPO/$IMAGE_NAME \
  --region $REGION
```

5. 部署及執行工作

在這個步驟中，您將建立具有直接虛擬私有雲輸出流量的作業，以便更快上傳至 Google Cloud Storage。

1. 建立 Cloud Run 工作：

```
gcloud run jobs create $JOB_NAME \  
  --region $REGION \  
  --image $REGION-docker.pkg.dev/$PROJECT_ID/$AR_REPO/$IMAGE_NAME \  
  --set-env-vars BUCKET_NAME=$BUCKET_NAME \  
  --set-secrets HF_TOKEN=$SECRET_ID:latest \  
  --cpu 8.0 \  
  --memory 32Gi \  
  --gpu 1 \  
  --add-volume name=finetuned_model,type=cloud-storage,bucket=$BUCKET_NAME \  
  --add-volume-mount volume=finetuned_model,mount-path=/finetune/new_model \  
  --service-account $SERVICE_ACCOUNT_ADDRESS
```

2. 執行工作：

```
gcloud run jobs execute $JOB_NAME --region $REGION --async
```

這項工作大約 10 分鐘就能完成。您可以透過上一個指令輸出內容中提供的連結，查看狀態。

6. 使用 Cloud Run 服務，透過 vLLM 提供微調模型

在這個步驟中，您將部署 Cloud Run 服務。這項設定會使用直接虛擬私有雲，透過私人網路存取 Cloud Storage 值區，加快下載速度。

- 部署 Cloud Run 服務：

```
gcloud run deploy serve-gemma-emoji \
  --image us-docker.pkg.dev/vertex-ai/vertex-vision-model-garden-dockers/pytorch-vllm-serve:20250601_0916_RC01 \
  --region $REGION \
  --port 8000 \
  --set-env-vars MODEL_ID=new_model,HF_HUB_OFFLINE=1 \
  --cpu 8.0 \
  --memory 32Gi \
  --gpu 1 \
  --add-volume name=finetuned_model,type=cloud-storage,bucket=$BUCKET_NAME \
  --add-volume-mount volume=finetuned_model,mount-path=/finetune/new_model \
  --service-account $SERVICE_ACCOUNT_ADDRESS \
  --max-instances 1 \
  --command python3 \
  --args="-m,vllm.entrypoints.api_server,--model=/finetune/new_model,--tensor-parallel-size=1" \
  --no-gpu-zonal-redundancy \
  --labels=dev-tutorial=codelab-tuning \
  --no-invoker-iam-check
```

7. 測試微調模型

在這個步驟中，您將使用 curl 提示模型，測試微調結果。

1. 取得 Cloud Run 服務的服務網址：

```
SERVICE_URL=$(gcloud run services describe serve-gemma-emoji \
  --region $REGION --format 'value(status.url)')
```

2. 為模型建立提示。

```
USER_PROMPT="Translate to emoji: I ate a banana for breakfast, later I'm thinking of
having soup!"
```

3. 使用 curl 呼叫服務來提示模型，並使用 jq 篩選結果：

```
curl -s -X POST ${SERVICE_URL}/v1/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: bearer $(gcloud auth print-identity-token)" \
-d @- <<EOF | jq ".choices[0].message.content"
{  "model": "${NEW_MODEL}",
  "messages": [{
    "role": "user",
    "content": [ { "type": "text", "text": "${USER_PROMPT}" } ]
  }]
}
EOF
```

畫面會顯示類似以下的回應：



步驟 8 · 約 0 分鐘

8. 恭喜！

恭喜您完成本程式碼研究室！

建議參閱 [Cloud Run Jobs GPU](#) 說明文件。

涵蓋內容

- 如何使用 Cloud Run Jobs GPU 進行微調
 - 如何使用 Cloud Run 和 vLLM 提供模型服務
 - 如何為 GPU 工作使用直接虛擬私有雲設定，加快模型上傳和服務速度
-

步驟 9 · 約 0 分鐘

9. 清除所用資源

為避免產生非預期費用 (例如 Cloud Run 服務遭非預期呼叫的次數，超過[免費層級每月 Cloud Run 呼叫配額](#))，您可以刪除步驟 6 中建立的 Cloud Run 服務。

如要刪除 Cloud Run 服務，請前往 Cloud Run Cloud 控制台 (<https://console.cloud.google.com/run>)，然後刪除 `serve-gemma-emoji` 服務。

如要刪除整個專案，請前往「管理資源」，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
